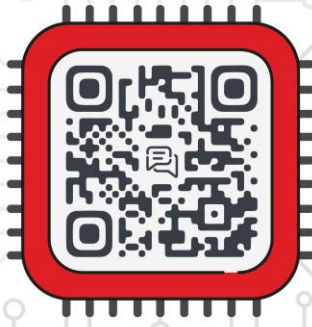




7



15



1800 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Graph



د. عملي مشترك

RB Informatics; 27/5/2025

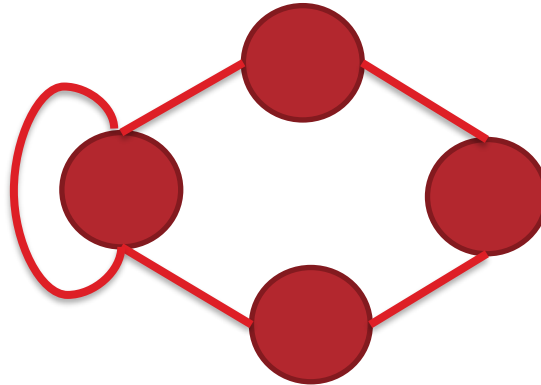
خوارزميات ٢

بعد أن تعرفنا في المحاضرات السابقة على أول بنية غير خطية والتي هي الشجرة، سنتعرف في هذه المحاضرة على نوع ثاني من البنى الغير خطية والذي هو البيان

So are you reaaaady to burn this lecture !!

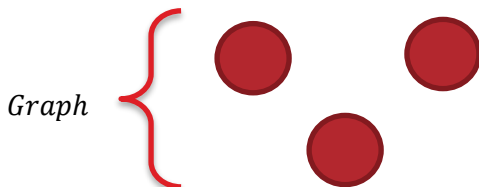
البيان Graph

هو بنية معطيات غير خطية تتألف من عقد vertex ووصلات Edges، والوصلة عبارة عن خط يصل بين عقدتين



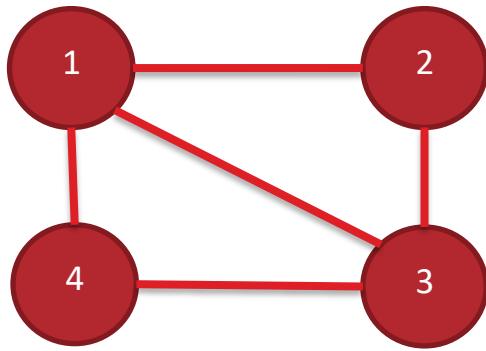
ملاحظات:

- ليس هناك قيود على البيان (لا وجود لمفهوم العقدة أو الجذر)
- ممكن أن تكون الوصلة من أي نوع
- ممكن عدم وجود أي وصلة
- ممكن للوصلة أن تكون بين العقدة ونفسها (المثال فوق)



أنواع البيان:

- بيان موجه Directed-Graph
- بيان غير موجه Undirected-Graph
- بيان موزون weighted-Graph
- بيان غير موزون Unweighted-Graph



ليكن لدينا هذا البيان الغير موجه:
ملاحظة: تكون وصلات البيان الغير
الموجه لا تحمل جهة، أي يمكن الانتقال
من (1) إلى (2) وبالعكس أيضاً.

يمكننا تمثيل البيان بالعديد من الطرق وسنتعرف على طريقتين منها:

1. مصفوفة ثنائية:

يكون عدد الأسطر يساوي عدد الأعمدة ويساوي عدد العقد
لنمثل البيان السابق:

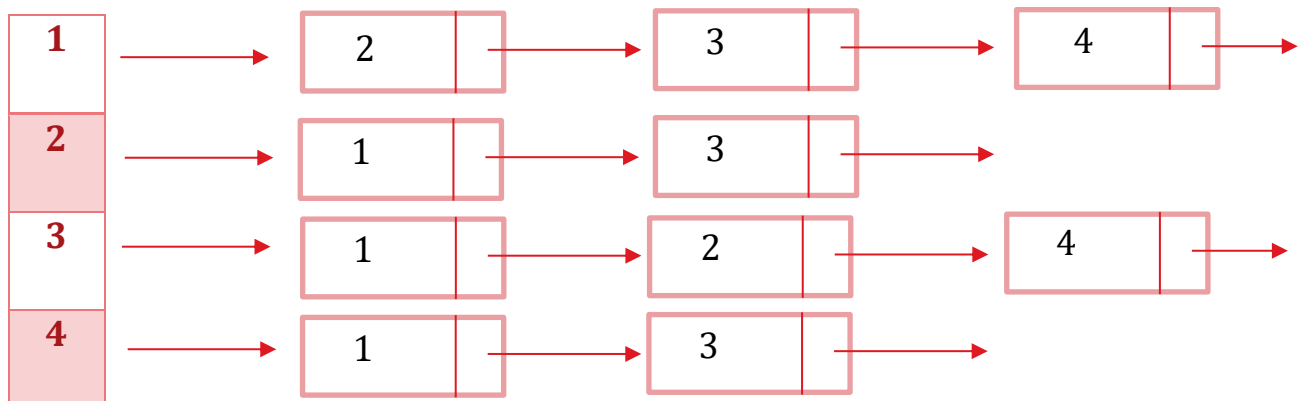
طريقة التمثيل:

عن طريق مقارنة إذا كان هناك وصلة بين عقدتين
أم لا، نضع (1) في حال كان هناك وصلة و (0)
في حال ليس هناك وصلة

	1	2	3	4
1	0	1	1	1
2	1	0	1	0
3	1	1	0	1
4	1	0	1	0

2. سلاسل التجاور:

سنقوم بحجز مصفوفة أحادية أو List، كل عنصر فيها يشير إلى list تحوي العناصر التي لديه وصلة معها
لنمثل البيان السابق:



ملاحظة: نقوم باستخدام نوع التمثيل حسب احتياجنا، فمثلاً لإيجاد العقد المجاورة لعقدة ما سلاسل
التجاور أفضل، أما لفحص وجود وصلة ما فالمصفوفة الثنائية أفضل وهكذا.

خوارزميات التجوال في البيان:

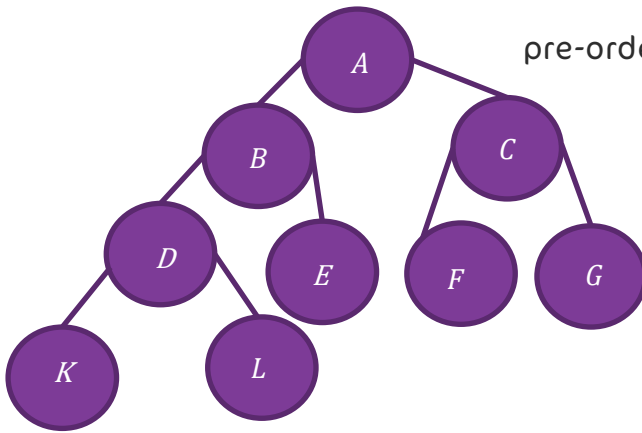
لدينا نوعين رئيسيين وهما

- البحث في العمق أولاً (Depth First search) DFS
- البحث في العرض أولاً (Breadth First search) BFS

أولاً: Depth First Search (DFS)

خوارزمية DFS شبيهة بخوارزميات التجوال المطبقة على الأشجار (Pre-Order-IN-Order-Post-Order) في هذه الخوارزمية نعلم العمق أولاً أي نذهب إلى أعظم عقدة في فرع ما من الشجرة أو من البيان حتى نصل إلى null فنقوم بالتراجع ونكرر العملية من أجل العقد الأخرى.

■ لنطبقها على الأشجار: سنستخدم DFS على أنها pre-order

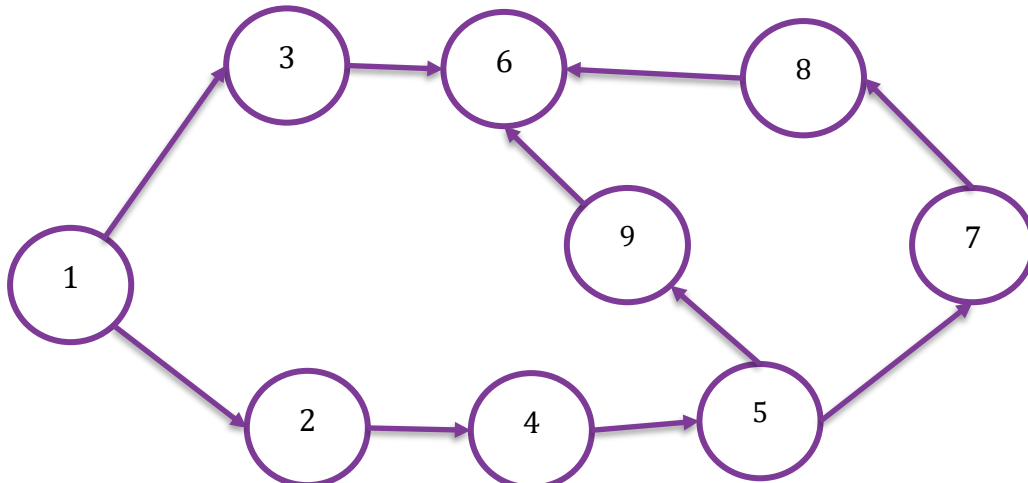


DFS: A B D K L E C F G

نلاحظ أننا استمرينا بالتجوال في الشجرة الفرعية اليسارية حتى وصلنا إلى null فقمنا بالتراجع ثم انتقلنا إلى الشجرة الفرعية اليمينية حتى وصلنا إلى null وتوقفنا لأنه تمت زيارة كل العقد

■ لنطبق خوارزمية DFS على هذا البيان الموجه:

ملاحظة: تكون وصلات البيان الموجه تحمل جهة؛ أي مثلاً في هذا البيان يمكننا الانتقال من 1 إلى 2 ولا يمكننا العكس



■ لنفترض أننا سنبدأ التجوال من العقدة (1):

سيكون لديها خيارين إما (2) أو (3) ولكن : إذا اعتمدنا التجوال على أنه pre-Order سيكون : العقدة ثم يسار ثم يمين، إذا تم قلب هذا البيان نرى أن 2 هي يسار 1

← ثم 4 يسار 2

1	2	4	...
---	---	---	-----

العقد المزارة:

← 5 يسار 4

1	2	4	5	...
---	---	---	---	-----

العقد المزارة:

← 7 يسار 5

1	2	4	5	7	...
---	---	---	---	---	-----

العقد المزارة:

← 8 يسار 7

1	2	4	5	7	8	...
---	---	---	---	---	---	-----

العقد المزارة:

← 6 يسار 8

1	2	4	5	7	8	6	...
---	---	---	---	---	---	---	-----

العقد المزارة:

هنا نتوقف !! لا يمكننا الإكمال إلى العقدة 3 لأنه ليس هناك وصلة من 6 إلى 3
نذهب للابن اليميني ل 6 والذي هو 9 ، ثم نعود إلى العقدة (1) من أجل ابنها اليميني (3)

DFS: 1,2,4,5,7,8,6,9,3

لنجرّب التجوال بطريقة أخرى: (عشوائية)

لنبدأ من (1): سيكون لديها خيارين إما (2) أو (3)
نختار (3):

العقد المزارة:

1	3	...
---	---	-----

ثم (6):

العقد المزارة:

1	3	6	...
---	---	---	-----

يمكننا أن نكمل بعد

هنا نتوقف لا

العقدة 6، إذا نتراجع للعقدة 3 فنرى أنها مزارة، نتراجع للعقدة 1 نرى أنها مزارة
ولكن يمكننا الذهاب من 1 إلى 2

العقد المزارة:

1	3	6	2	...
---	---	---	---	-----

ثم 4:

1	3	6	2	4	...
---	---	---	---	---	-----

ثم 5:

1	3	6	2	4	5	...
---	---	---	---	---	---	-----

لدي 5 خيارين: إما 7 أو 9 سنختار 7 (يمكننا اختيار 9)

1	3	6	2	4	5	7	...
---	---	---	---	---	---	---	-----

ثم 8:

1	3	6	2	4	5	7	8	...
---	---	---	---	---	---	---	---	-----

ثم 6 (نرى أنها مزارة)

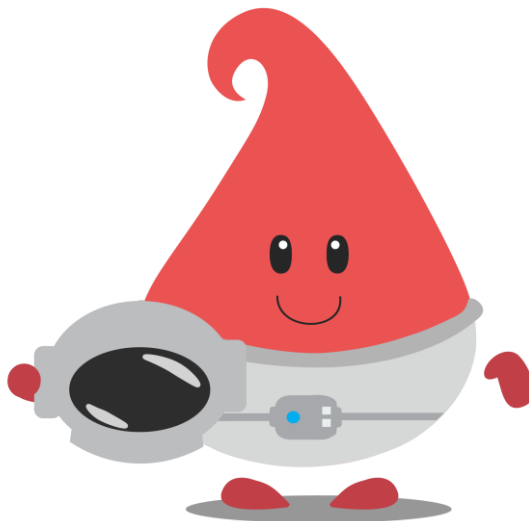
نتراجع ل 7 (نرى أنها مزارة)

نتراجع ل 5 (نرى أنها مزارة) ولكن يمكننا الذهاب من 5 الى 9

العقد المزارة

1	3	6	2	4	5	7	8	9
---	---	---	---	---	---	---	---	---

ملاحظة: نرى أننا في الحالتين زرنا جميع العقد ولكن من خلال مبدأ العمق أولاً

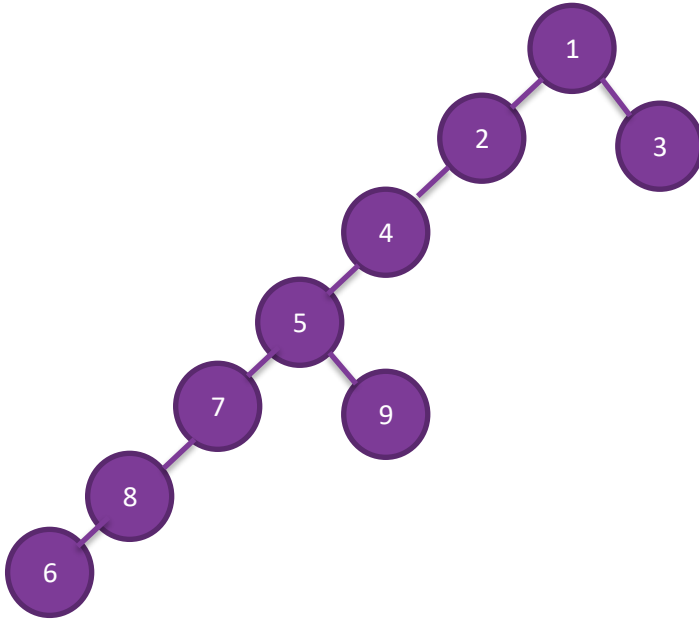
.Prove yourself to yourself not others

سؤال هام: مثل مخطط البيان السابق كشجرة

سنأخذ الحالة الأولى للسهولة (عندما تجولنا حسب مبدأ الـ pre-order):

1	2	4	5	7	8	6	9	3
---	---	---	---	---	---	---	---	---

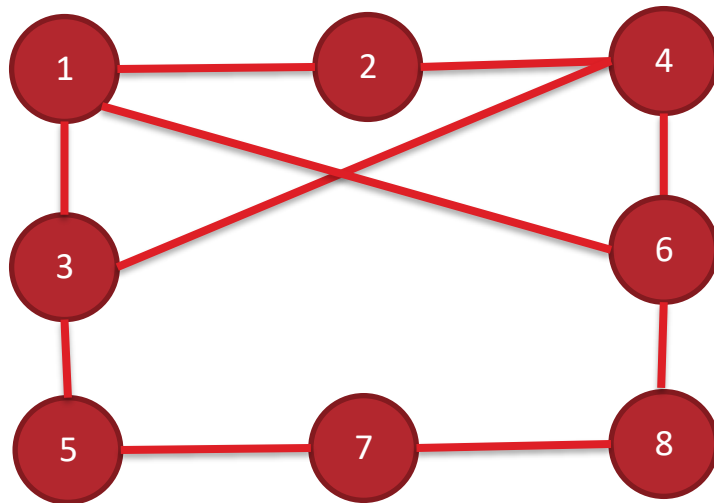
مصفوفة العقد المزارة:



مثال 2: ليكن لدينا هذا البيان ونريد التجول عليه بخوارزمية DFS:

نرى أنه غير موجه وغير موزن نبدأ من العقدة 1 ثم القسم اليساري (جميع الطرق المتاحة)

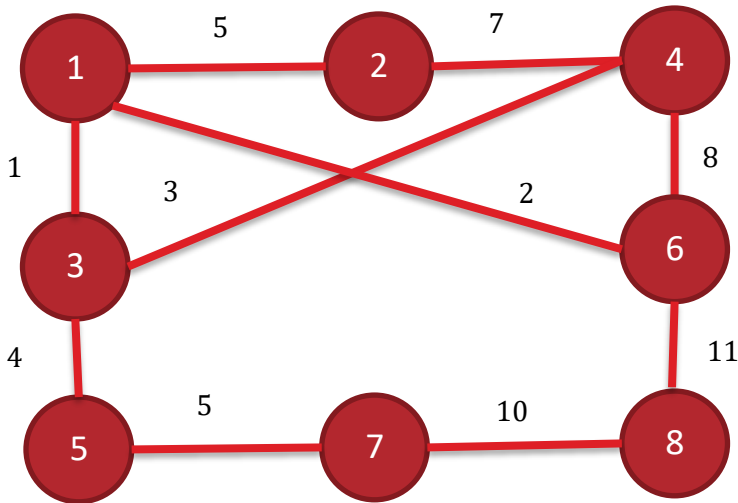
ملاحظة : إن لم يتم تحديد مكان البدء من نص السؤال، نبدأ من العقدة الأصغر، كما هنا بدأنا من (1) ولكن في النظري تُذكر العقدة التي سنبدأ منها صراحةً في نص السؤال



1	3	5	7	8	6	4	2
---	---	---	---	---	---	---	---

مثال 3: ليكن لدينا هذا البيان ونريد التجول عليه بخوارزمية DFS:
نرى أن هذا البيان موزون وغير موجه

ملاحظة: في البيان الموزون تحمل الوصلة بيانات معينة.



آلية التجوال في البيان الموزون:

نبدأ من العقدة (1) سيكون لدينا خيارين إما 2 أو 3، في البيان الموزون سنسلك الطريق الأقل وزناً (أي الأقل تكلفة)

سنسلك 3:

العقد المزاردة : 1 3

أمام (3) خيارين إما 4 أو 5
نسلك 4 صاحبة الوزن الأقل

العقد المزاردة : 1 3 4

أمام (4) ثلاث خيارات : 2 و 3 و 6
3 (صاحبة الوزن الأقل) مزاردة , تبقى 2 و 6
نسلك (2) صاحبة الوزن الأقل

العقد المزاردة : 1 3 4 2

أمام (2) خيارين : 4 و 1 ولكن هذه العقد مزارة، إذا نعود للخطوة السابقة عندما كنا عند العقدة 4
نسلك (6)

1	3	4	2	6	
---	---	---	---	---	-------

العقد المزارة :

أمام (6) خيارين (1) و (8)، (1) صاحبة الوزن الأقل ولكنها مزارة
نسلك 8

العقد المزارة:

1	3	4	2	6	8	
---	---	---	---	---	---	-------

أمام 8 خيار واحد (7)
نسلك 7

العقد المزارة

1	3	4	2	6	8	7	
---	---	---	---	---	---	---	-------

تبقى 5

نسلك 5

العقد المزارة:

1	3	4	2	6	8	7	5
---	---	---	---	---	---	---	---

الخوارزمية:

```

1 void DFS (int v)
2 {
3     boolean [ ] visited = new boolean [n];
4     for (int i = 0; i < n; i++) visited [i] = false;
5     DFSUtil (v, visited);
6 }
7 void DFSUtil (int v, boolean [ ] visited)
8 {
9     visited[v] = true; System.out.print(v + " ");
10    for (Integer vertex : adjacencyList.get(v))
11    {
12        if ( !visited [vertex] )
13        DFSUtil(vertex, visited); }}
```



شرح الخوارزمية :

- السطر 1: يستقبل تابع DFS وسيط واحد وهو العقدة التي سنبدأ التجول منها وفرضاً التتابع في كلاس ال graph بحيث أن n هو عدد العقد global variable
- السطر 3: نعرف مصفوفة من النوع boolean (من أجل تحديد العقد المزاراة ب true والغير مزاراة ب false).
- السطر 4: نجعل القيمة الابتدائية لجميع العقد هي false
- السطر 5: نستدعي التابع DFSUtil من أجل الوسيطين: العقدة التي سنبدأ التجول منها ومصفوفة ال visited
- السطر 9: نجعل العقدة التي نقف عندها على أنها مزاراة
- السطر 10 ← 13: نمر على باقي عقد المصفوفة ونفحص إذا كانت false (أي غير مزاراة) نعيد استدعاء التابع DFSUtil من أجلها، وهكذا حتى تنتهي

(التعقيد: $O(V + E)$ لأننا سوف نمر على جميع العقد والوصلات وعلى الأكثر سنزورها مرة واحدة)

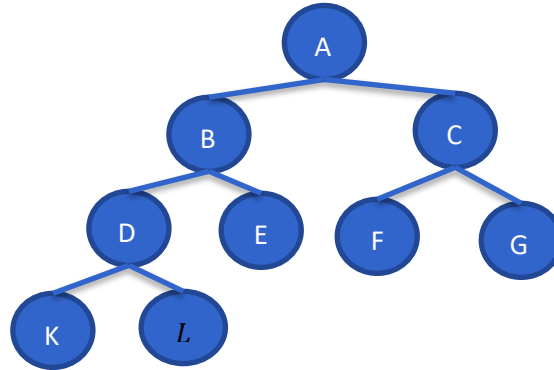
وما أرى استنزاف طاقتي لطلب العلم إلا جواباً مهذباً لـ:

" وعن شبابه فيما أفناه " 🐦



خوارزمية Breadth First Search (BFS)

يقوم أساس هذه الخوارزمية على البدء من عقدة معينة ومعالجتها ومن ثم الانتقال إلى ابنها الأول ومعالجته ومن ثم ابنها الثاني والثالث وهكذا، ثم أبناء الابن الأول وأبناء الابن الثاني والثالث (ننتقل مستوى مستوى) لنجرب تطبيقها على الشجرة:



نبدأ من A:

نأخذ أبناءها: BC

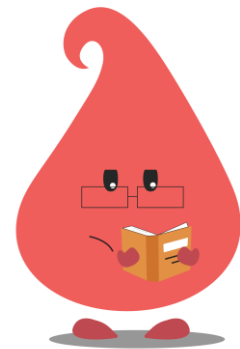
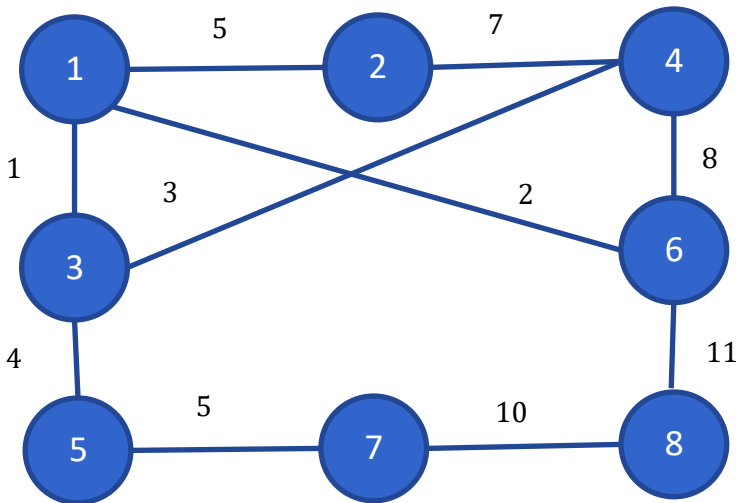
نأخذ أبناء B: DE

نأخذ أبناء C: FG

نأخذ أبناء D: KL

ABCDEFGKL

■ مثال : ليكن لدينا هذا البيان (غير موجه + موزون) ونريد تطبيق خوارزمية BFS عليه



نبدأ من العقدة 1 :

نأخذ أبناءها (3,2,6) كيف سيتم ترتيبهم؟؟

بما أنه بيان موزون نأخذ الطريق الأقل وزناً

العقد المزارة :

1	3	6	2	...
---	---	---	---	-----

■ نأخذ أبناء العقدة 3: (5,4) أيضاً بالترتيب حسب الطريق الأقل وزناً

العقد المزارة :

1	3	6	2	4	5	...
---	---	---	---	---	---	-----

- نأخذ أبناء العقدة 6: (1,4,8) العقدين 1 و 4 مزارتين , إذا نأخذ العقدة 8 العقد المزاراة :

1	3	6	2	4	5	8	...	...	...
---	---	---	---	---	---	---	-----	-----	-----

- نأخذ أبناء العقدة 2: نرى أنهم عقد مزاراة (4,1)
- نأخذ أبناء العقدة 4: نرى أنهم عقد مزاراة (2,3,6)
- نأخذ أبناء العقدة 5 : (3,7) نرى أن 3 مزاراة والعقدة 7 غير مزاراة إذاً نأخذها العقد المزاراة

1	3	6	2	4	5	8	7
---	---	---	---	---	---	---	---

- انتهينا من زيارة جميع العقد.

ملاحظة: إذا كان البيان غير موزون ونريد تطبيق خوارزمية BFS عليه نأخذ الأولاد: يسار -يمين أو العكس صحيح

الخوارزمية:

```

1 public void BFS(int s) {
2     boolean [ ] visited = new boolean [n];
3     for (int i = 0; i < n; i++) visited [i] = false;
4     Queue q = new LinkedList< >( );
5     q.add(s);
6     visited [s] = true;
7     while (!q.isEmpty( ))
8     {
9         Integer v = q.poll( );
10        System.out.print(v + " ");
11        for (Integer i : adjacencyList.get(v)) {
12            {
13                if (!visited[i])
14                {
15                    visited[i] = true;
16                    q.add(i);
17                }
18            }
19        }
20    }
21 }
```



شرح الخوارزمية :

السطر 1: يستقبل تابع الـ BFS وسيط واحد هو العقدة التي سنبدأ منها وفرضاً التتابع في كلاس الـ graph بحيث أن n هو عدد العقد global variable

السطر 2+3: نعرف مصفوفة العقد المزارة من النوع Boolean ونجعل القيمة الابتدائية لها هي false

السطر 4: سنستخدم queue لتطبق مبدأ First IN First Out

السطر 5+6: نضيف أول عقدة إلى الـ queue ونجعلها مزارة

السطر 7 ← 9: نحذف العنصر في بداية الـ queue

السطر 11 ← 16: نمر على جميع العقد ونفحصها إن لم تكن مزارة نجعلها مزارة، ثم نضيفها إلى الـ queue

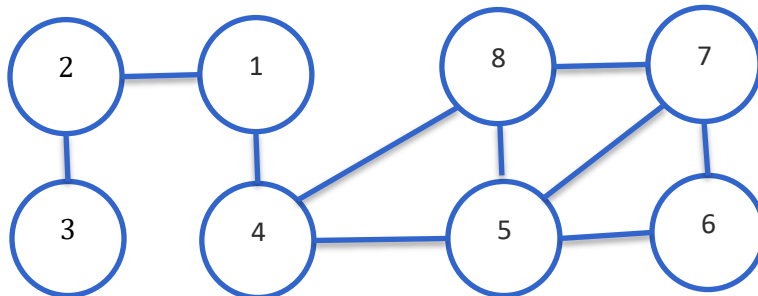
ملاحظة: لاحظ أن العقد التي زرتها أولاً نمر جيرانها أولاً أي هنالك نوع من الدور والترتيب، لذلك سنستخدم queue والذي ينطبق مبدأ FIFO (First IN First Out)

Shortest Path in unweighted

سؤال: أي الخوارزميتين DFS, BFS لها أقصر مسار؟

في خوارزمية DFS، كنا نمر العقد بشكل عشوائي حتى تتم زيارتها جميعاً
بينما من خوارزمية BFS، نمر العقد التي تبعد مسافة واحدة عن المبدأ أولاً، ثم العقد التي تبعد مسافة واحدة عن المبدأ أولاً، ثم العقد التي تبعد مسافة وصليتين، ثم العقد التي تبعد مسافة ثلاث وصلات وهكذا
لذلك إن أول ظهور للعقدة ضمن عملية التجوال BFS هو أقصر طريق مبدأ إلى هذه العقدة
إذا سنعتمد خوارزمية BFS في حساب أقصر طريق من عقدة إلى عقدة

مثال : ما هو أقصر طريق للوصول من العقدة 1 إلى 8؟



1. أولاً سنتعامل مع queue كما تعلمنا:

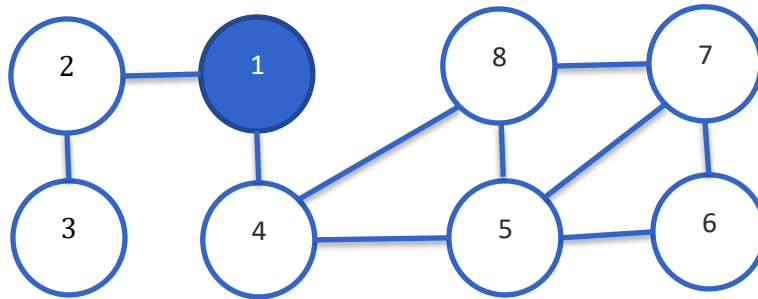
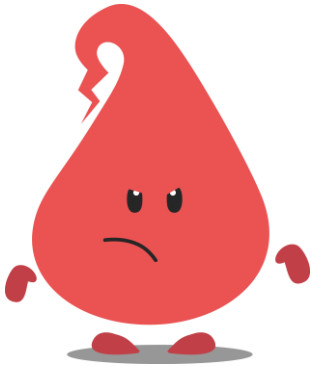
نبدأ من العقدة 1:



2. ثانياً: سيكون لدينا ثلاث مصفوفات لتساعدنا في إيجاد أقصر طريق وهي:

	1	2	3	4	5	6	7	8
visited	0	0	0	0	0	0	0	0
distance	-1	-1	-1	-1	-1	-1	-1	-1
predecessor	-1	-1	-1	-1	-1	-1	-1	-1

- Visited : مصفوفة العقد المزارة (القيمة الابتدائية لها (0)
- Distance : مصفوفة تحوي طول المسار (عدد الوصلات)
- Predecessor : مصفوفة تحوي العقدة التي أتينا منها إلى عقدة ما



We are almost done

■ لنكمل:

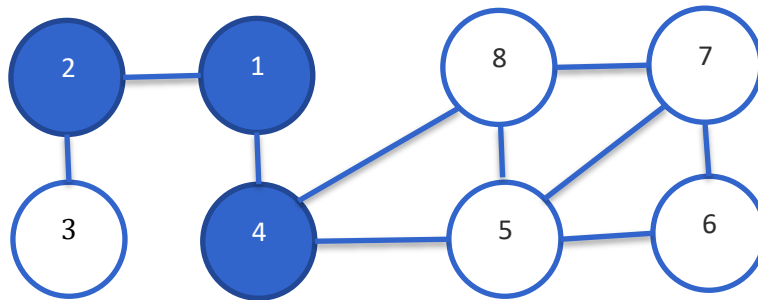
بعدما بدأ من 1 أصبحت المصفوفات

	1	2	3	4	5	6	7	8
visited	1	0	0	0	0	0	0	0
distance	0	-1	-1	-1	-1	-1	-1	-1
predecessor	-1	-1	-1	-1	-1	-1	-1	-1

■ الآن نزور جيران العقدة 1 أي (2,4) العقدة 2, 4 مزارتين المسافة بين العقد 1 والعقدتين 2 و4 هي 1

	1	2	3	4	5	6	7	8
visited	1	1	0	1	0	0	0	0
distance	0	1	-1	1	-1	-1	-1	-1
predecessor	-1	1	-1	1	-1	-1	-1	-1

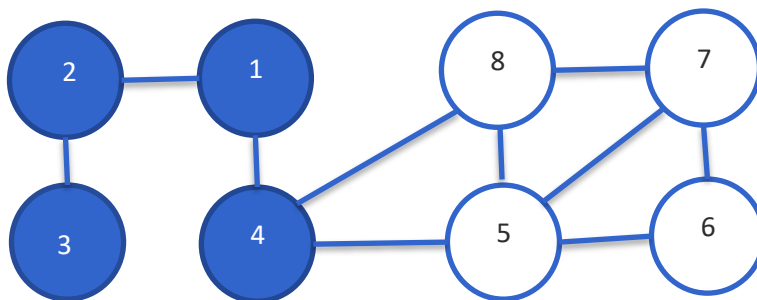
2	4							
---	---	--	--	--	--	--	--	--



■ نزور جيران (2): العقدة 3

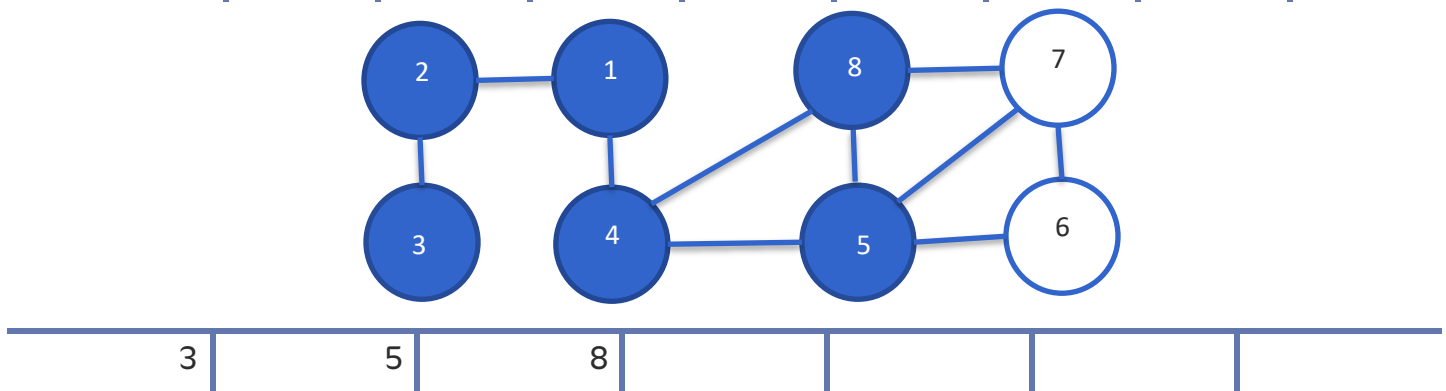
	1	2	3	4	5	6	7	8
visited	1	1	1	1	0	0	0	0
distance	0	1	2	1	-1	-1	-1	-1
predecessor	-1	1	2	1	-1	-1	-1	-1

4	3							
---	---	--	--	--	--	--	--	--



■ نزور جيران العقدة (4) (5,8)

	1	2	3	4	5	6	7	8
visited	1	1	1	1	1	0	0	1
distance	0	1	2	1	2	-1	-1	2
predecessor	-1	1	2	1	4	-1	-1	4



3	5	8						
---	---	---	--	--	--	--	--	--

■ ها قد وصلنا إلى العقدة 8

أقصر طريق للوصول لها هو $1 \leftarrow 4 \leftarrow 8$ والمسافة هي 2

ولكن ما فائدة مصفوفة ال predecessor؟ في حال أردنا طباعة أقصر طريق من 1 إلى 8 نقوم بطباعة خطوة، خطوة من

خلال التراجع إلى كل عقدة

أي هنا في مصفوفة predecessor عند عقدة 8 قيمتها 4 (نتراجع إلى 4) عند 4 قيمتها 1 (نتراجع إلى 1) عند 1 قيمتها -

1, نتوقف ويكون الخرج [1-4-8] الذي هو أقصر طريق

The End

في الحياة ما عليك إلا السعي والله يتكفل بأمرك، لا
تفكر كيف سيأتيك الفرغ، تدابير الله عظيمة وهو
القدير العزيز، لا تستبق الأحداث وتخاف الابتلاء، فكل
بلاء لطفه معه ورحمة الله ترافقه، لا أحد يعرفك من
الداخل إلا الله فلا تجزع وتقلق، ما عليك إلا السعي
والإيمان أنه بك أرحم الراحمين. 🍀 ❤️